

HW1:

Please write a correct programmer/manager class in C++

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
class Employee{
```

```
public:
```

```
    Employee(char *name, int id);
```

```
    Employee(const Employee &other); // Add copy constructor
```

```
    Employee& operator=(const Employee &other); // Add assignment operator
```

```
    ~Employee();
```

```
    char *getName() { return _name; };
```

```
private:
```

```
    int _id;
```

```
    char *_name;
```

```
};
```

```
Employee::Employee(char *name, int id){
```

```
    _id = id;
```

```
    _name = new char[strlen(name) + 1];
```

```
    strcpy(_name, name);
```

```
}
```

```
Employee::~~Employee(){
```

```
    delete[] _name;
```

```
}
```

```

// Copy constructor copies the data from another Employee object
// not using the default shallow copy.
Employee::Employee(const Employee &other ){
    _id = other._id;
    _name = new char[strlen(other._name) + 1];
    strcpy(_name, other._name);
}

// Assignment operator, if needed. make sure no dinging pointer and memory leak.
Employee& Employee::operator=(const Employee &other) {
    if (this != &other) {
        _id = other._id;
        delete[] _name;
        _name = new char[strlen(other._name) + 1];
        strcpy(_name, other._name);
    }
    return *this;
}

int main(){
    Employee *programmer = new Employee("John", 22);
    cout << programmer->getName() << endl;

    // Lots of code ....

    Employee manager = *programmer;
    // Creates a new Employee "manager",
    // which is an exact copy of the
    // Employee "programmer".

```

```
delete programmer;
```

```
cout << manager.getName() << endl;
```

```
return 0;
```

```
}
```

HW2:

假設有以下兩個 class，請幫我們實作 Foo 類別的解構子，複製建構子和賦值運算子 (相信我，這題不像看起來的那麼簡單，請考慮 exception-safe和memory leak的各種可能問題)

(參考：<http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-ctor>)

```
#include

class Bar {
private:
    std::array not_important_;
};

class Foo {
public:
    Foo()
        : bar_{new Bar{}} {}

    // Rule of three!
    Foo(const Foo& rhs) {
        if (rhs.bar_) {
            bar_ = new Bar{*rhs.bar_};
        }
    }

    Foo&
    operator=(const Foo& rhs) {
        if (this != &rhs) {
            if (rhs.bar_) {
                Bar* new_bar = new Bar{*rhs.bar_};
                delete bar_;
                bar_ = new_bar;
            } else {
                delete bar_;
                bar_ = nullptr;
            }
        }
        return *this;
    }

    ~Foo() {
        delete bar_;
    }
}
```

```
private:
    // XXX: Well, this is not doing well in practical...
    // but hey, this is just some exercise. Please use
    // std::unique_ptr or std::shared_ptr instead.
    //
    // The Foo object will "own" this Bar pointer.
    Bar* bar_{nullptr};
};
```